

Extending the MultiLexNorm++ Pipeline with Length-Aware Detection and Language-Specific Threshold Optimization

Yujiaxuan Wang

Technical University of Munich
ge84qof@mytum.de

Jiayang Zhang

Technical University of Munich
ge65nus@mytum.de

Abstract

This work improves a multi-lexical normalization pipeline using language-specific threshold optimization and a Length-Aware Detector. The detector predicts whether a token should be kept or normalized into one, two, or three or more words. We evaluate the pipeline on 12 languages with three LLMs. The complete pipeline improves average ERR and F1 over the reproduced baseline by 3.79 and 1.79 percentage points, with particularly strong gains on several Asian languages. Ablation experiments show that threshold optimization is the main source of improvement, while the Length-Aware Detector provides smaller, language- and model-dependent benefits.

1 Introduction

Social media such as Twitter or Instagram provides us a large amount of user-generated data. However, much of these texts is written in a non-standard style, including abbreviations, capitalization errors, spelling errors, and other informal variants. Such noise often reduces the performance of NLP models. Lexical normalization aims to transform non-standard words into their canonical forms, including one-to-many (1-n) and many-to-one (n-1) replacements (Han and Baldwin, 2011; van der Goot et al., 2021a). Using normalized text as input data can benefit downstream NLP tasks such as intent classification (Zhang et al., 2022) and named entity recognition (Plank et al., 2020).

The MultiLexNorm shared task is a large publicly available multilingual benchmark for lexical normalization, covering 12 languages (van der Goot et al., 2021a). However, it mainly focuses on Indo-European languages written in Latin script.

MultiLexNorm++ extends this to five Asian languages from different language families in four different scripts (Buaphet et al., 2026). It also proposes a three-stage lexical normalization pipeline.

A detector model first labels words with high confidence as candidates for normalization. In low-entropy cases, detected tokens are replaced through lookup in the MFR dictionary, while the remaining candidates are passed to the LLM-based component.

However, there are two limitations in the original pipeline. First, binary detection only predicts whether a word should be normalized, but ignores the expected output length, which is especially relevant for several Asian languages with more frequent one-to-many replacements. Second, a single fixed threshold setting of detector and dictionary may be suboptimal across languages with different error word distributions and detector performance.

To address these issues, we reproduce the original MultiLexNorm++ detector-dictionary-LLM pipeline and investigate two extensions: length-aware detection and language-specific threshold optimization.

Our main contributions are: We reproduce the original MultiLexNorm++ detector-dictionary-LLM pipeline; extend the original binary detector to a length-aware detector; perform language-specific threshold optimization; We analyze effect of both extensions through ablation experiments

2 Method

2.1 Reproduced Baseline Pipeline

We first reproduce the original MultiLexNorm++ detector-dictionary-LLM pipeline as our baseline (Buaphet et al., 2026). We construct and run the full pipeline separately for each language. The pipeline consists of three stages: normalization detection, MFR dictionary replacement, and LLM-based normalization.

The dataset provides aligned pairs of raw and normalized sentences, where each sentence is divided into word-level chunks. We split the training data into a training split and a development split.

For detector training, we convert each aligned raw-normalized word pair into a binary label. If the raw word differs from the normalized form, this word is labeled as NORM. Otherwise, it is labeled as KEEP.

In the first stage, we trained the detector model on the prepared training data. For each input word, the detector predicts whether it should be kept or normalized, together with a confidence score. The predicted label with a higher confidence score is selected as the output. This is equivalent to using 0.5 as the default detector threshold.

In the second stage, we construct an MFR dictionary from the training data. This records raw words that are mapped to at least one different normalized form. For each raw word, we count the frequency of its possible normalization mappings and compute the corresponding Miller-Madow entropy. Low-entropy candidates are considered as reliable and used for replacement of raw words. Since the original paper does not specify the default entropy threshold, we use 0.5 as the default entropy threshold in our reproduced baseline.

In the final stage, all remaining detected words are passed to the LLM-based component. For each candidate, we construct a prompt containing the detected word, its surrounding context, and few-shot examples sampled from the training data. The LLM is then expected to predict the normalized form of the candidate.

2.2 Length-Aware Detector

The original detector formulates normalization detection as a binary classification, which assigns each input token either KEEP (implemented as 0 in real task) or NORM. Since some tokens, especially frequent in Asian languages, need normalization from one word to multiple words (1-n normalization). We extend this detection with a length-aware label space: $\mathcal{Y} = \{\text{KEEP}, \text{NORM_1WORD}, \text{NORM_2WORD}, \text{NORM_3PLUS}\}$. The introduction of multiple normalization labels also requires a modification to the calculation of detector confidence. In the original binary detector, a token is selected for normalization when the confidence assigned to NORM exceeds the detector threshold. In the length-aware detector, the overall normalization confidence is instead defined as the sum of the probabilities assigned to all non-KEEP labels: $p_{\text{norm}}(x) = p(\text{NORM_1WORD} | x) + p(\text{NORM_2WORD} | x) + p(\text{NORM_3PLUS} | x)$. Then a token is passed to the normalization

pipelines when $p_{\text{norm}}(x) \geq \tau_{\text{det}}$, where τ_{det} is the detector threshold. If the threshold is satisfied, the predicted length category is selected as the highest-confidence non-KEEP label, otherwise the token is classified as KEEP

$$\hat{y} = \underset{y \in \{\text{NORM_1WORD}, \text{NORM_2WORD}, \text{NORM_3PLUS}\}}{\arg \max} p(y | x). \quad (1)$$

This formulation allows the detector not only identify whether normalization is required but also estimate the expected length category of the normalized output. The predicted category can subsequently be used as a length-related prompt constraint for downstream LLM normalization: NORM_1WORD denotes a one-word normalization, NORM_2WORD denotes a two-word normalization, and NORM_3PLUS denotes a normalization containing three or more words. Summing the non-KEEP probabilities prevents conservative normalization when probabilities are distributed across multiple plausible length categories, while the maximum-probability label retains a concrete length prediction for downstream processing.

2.3 Language-Specific Threshold Optimization

The reproduced baseline pipeline contains two important thresholds: the detector threshold and the entropy threshold. The detector threshold controls which words are selected as normalization candidates. A lower threshold increases the recall range. The entropy threshold controls whether an MFR dictionary candidate is reliable for normalization replacement. A lower threshold decrease the number of replacement by dictionary and sends more candidates to the LLM stage.

We perform a 5×6 grid search over detector and entropy thresholds. The detector threshold is selected from $\{0.1, 0.3, 0.5, 0.7, 0.9\}$, and the entropy threshold is selected from $\{0.2, 0.5, 0.8, 1.1, 1.4, 1.7\}$. To reduce repeated LLM inference cost, we first run the pipeline with the lowest threshold combination to maximize the range of LLM candidates. The LLM outputs are cached and then reused for later threshold combinations.

After running all combinations, the thresholds are selected primarily based on the best Error Reduction Rate (ERR) on the development set. However, when the ERR improvement is negligible, we use F1 as a secondary metric to avoid settings that harm the precision-recall balance. If multiple

settings have similar performance, we prefer the configuration closer to the default threshold of 0.5. We additionally inspect the development curves as a sanity check to avoid unstable choices.

3 Experimental Setup

3.1 Dataset

We use the MultiLexNorm++ dataset for all experiments (Buaphet et al., 2026). The full dataset contains 17 languages. Since validation data is unavailable for five languages, we conduct our experiments on 12 languages, including 7 Indo-European languages from the original MultiLexNorm benchmark, and 5 new Asian languages introduced by MultiLexNorm++.

The training data contains 35.9k rows. We split it into a 90% internal training split and a 10% development split. The internal training split is used to train the detector, construct the MFR dictionary, and sample few-shot examples for LLM prompting. The development split is used as the validation split during detector training and later for threshold selection. The validation data contains 8.4k rows and is used for final evaluation.

3.2 Models

For the normalization detector, we use an XLM-Roberta-based transformer model (Conneau et al., 2020). We follow the training setup described in the MultiLexNorm++ and use the standard hyperparameters of MaChAmp (van der Goot et al., 2021b).

For the LLM-based normalization stage, we use Qwen2.5:7b, Qwen3.5:9b and Deepseek-V4-Pro. We set the temperature to 0 and disable thinking mode to make the outputs more deterministic. The LLM receives the detected candidate word, its surrounding sentence context, and 8-shot examples sampled from the training data. The complete training configuration and prompt templates are provided in the Appendix.

3.3 Ablation Settings

To separate the effects of length-aware detector and threshold optimization, we use a 2×2 ablation design. The first dimension is detector type: original binary detector or length-aware detector. The second dimension is threshold setting: default thresholds or language-specific optimized thresholds. The reproduced MultiLexNorm++ pipeline with the original detector and default thresholds is used as baseline for our experiment. Ablation 1

evaluates the length-aware detector without threshold optimization, while Ablation 2 retains the original detector and applies threshold optimization only. Ablations 3 and 4 further examine the interaction between the two components, the former compares the two detectors under optimized thresholds and the latter measure the effect of threshold optimization on the length-aware detector only.

3.4 Evaluation Metrics

Since most words in the dataset do not require normalization, raw accuracy could be very high. Thus the accuracy does not reflect the system’s ability to perform normalization. Instead, we use Error Reduction Rate (ERR) as the primary metric to evaluate the pipeline. ERR measures how many errors the system reduces compared with the leave-as-is baseline, where the raw input is directly kept unchanged. A higher ERR indicates a better result. We use F1 as a secondary metric. F1 measures the balance between precision and recall. This reflects whether the normalization changes are too conservative or too aggressive. A higher F1 indicates a better precision-recall balance.

4 Results

4.1 Results of new pipeline

Figure 1 presents the results obtained with our new pipeline and Qwen3.5-9B. To conserve space, we report the complete language-level comparison for this model only, while results with the other two LLM are placed in Appendix. *Baseline* denotes reproduction of the original pipeline, which uses the original detector without threshold search, while *New Pipeline* combines the new length-aware detector with language-specific threshold optimization. ERR, F1 and differences (Δ) between two groups are reported as percentages. The five Asian languages: Indonesian (ID), Japanese (JA), Korean (KO), Thai (TH), and Vietnamese (VI) are highlighted by underlining their corresponding improvements.

The new pipeline improves the averaged ERR and F1 of Qwen3.5 by 3.34% and 1.93% respectively. ERR increases on 10 of the 12 languages. Particularly large gains are observed on Asian languages TH (+9.41 ERR), KO (+9.04), and VI (+7.68), while JA also shows a positive but smaller improvement of 2.63. The largest F1 gains are observed on TH (+14.19) and VI (+4.97). The negative ERR changes are limited to DE (-1.03) and

IDEN (-1.37), indicating that the occasional degradation in normalization accuracy is relatively small and limited. F1 also slightly decreases on DE, HR, IDEN (maximal -3.14 on KO)

The same overall trend is observed with the other two LLMs. Qwen2.5 obtains the largest average improvement (Figure 6), with gains of 6.14 ERR and 2.56 F1. Notably, its ERR improves on all 12 languages. The largest ERR gains again occur on several Asian languages, which are KO (+14.46), VI (+11.49), TH (+10.94), and JA (+7.32). Its F1 also improves on 11 of the 12 languages, with KO being the only exception. DeepSeekV4-Pro shows smaller but still positive gains of 1.89 ERR and 0.88 F1. Its most pronounced improvements include JA (+8.64 ERR and +5.83 F1) and VI (+3.32 ERR and +2.06 F1).

Among all the evaluated models. The smallest Qwen2.5-7b benefits the most from new pipeline. The improvement decreases from Qwen2.5-7b to Qwen3.5-9b and further to DeepSeekV4-Pro, suggesting that the length-aware detector and threshold optimization are particularly beneficial when the downstream LLM has more limited capacity.

Language	Baseline		New Pipeline		$\Delta \text{ERR} / \Delta \text{F1}$
DE	49.03	69.71	48.00	67.66	-1.03 / -2.05
EN	56.56	76.64	60.82	79.50	4.26 / 2.86
HR	47.89	68.73	49.14	68.34	1.25 / -0.39
ID	62.76	77.69	64.03	78.58	<u>1.27 / 0.89</u>
IDEN	60.17	78.64	58.80	77.75	-1.37 / -0.89
JA	14.35	37.43	16.98	37.62	<u>2.63 / 0.19</u>
KO	-9.04	20.96	0.00	17.82	<u>9.04 / -3.14</u>
NL	38.52	61.20	40.81	63.47	2.29 / 2.27
SL	54.62	73.30	58.18	76.56	3.56 / 3.26
SR	56.43	74.16	57.46	75.20	1.03 / 1.04
TH	24.05	50.78	33.46	64.97	<u>9.41 / 14.19</u>
VI	68.03	81.83	75.71	86.80	<u>7.68 / 4.97</u>
Avg Δ	-	-	+3.34	+1.93	-

Figure 1: Final result of new pipeline with Qwen3.5:9b

4.2 Ablation 1: Length-Aware Detector

Ablation 1 isolates the contribution of the length-aware detector by comparing the reproduced baseline with a variant, where only the detector is replaced and the default thresholds are retained. The resulting average changes are small across all three LLMs. For Qwen3.5, ERR and F1 decrease by 0.44% and 0.92%. Qwen2.5 (Figure 9) exhibits a mixed but similarly change: ERR increases by 0.44, while F1 decreases by 0.44. DeepSeekV4-Pro (Figure 11) shows a slight improvement of 0.57 ERR, while its F1 remains effectively unchanged at +0.02.

Averaged across the three models, replacing the detector only results in a limited ERR improvement

Language	Baseline ERR/F1		Ablation1 ERR/F1		$\Delta \text{ERR} / \Delta \text{F1}$
DE	49.03	69.71	47.08	67.09	-1.95 / -2.62
EN	56.56	76.64	58.45	77.95	1.89 / 1.31
HR	47.89	68.73	48.84	68.66	0.95 / -0.07
ID	62.76	77.69	63.78	78.32	<u>1.02 / 0.63</u>
IDEN	60.17	78.64	54.70	75.97	-5.47 / -2.67
JA	14.35	37.43	16.98	37.62	<u>2.63 / 0.19</u>
KO	-9.04	20.96	-6.02	14.56	<u>3.02 / -6.40</u>
NL	38.52	61.20	36.14	59.18	-2.38 / -2.02
SL	54.62	73.30	54.16	72.95	-0.46 / -0.35
SR	56.43	74.16	57.24	75.04	0.81 / 0.88
TH	24.05	50.78	17.56	50.08	<u>-6.49 / -0.70</u>
VI	68.03	81.83	69.15	82.58	<u>1.12 / 0.75</u>
Avg Δ	-	-	-0.44	-0.92	-

Figure 2: Ablation 1 with Qwen3.5:9b: Effect of length-aware detector only

of +0.19 and an F1 reduction of -0.45. Therefore, the length-aware detector does not provide neither a significant improvement nor a consistent degradation. The magnitude of both its positive and negative effects remains small, suggesting that the detector replacement does not substantially destabilize the original pipeline.

4.3 Ablation 2: Threshold Optimization

Ablation 2 isolates the contribution of threshold optimization by retaining the original detector and applying language-specific optimized thresholds. Compared with the baseline, threshold optimization produces significant improvements for both Qwen models. Qwen2.5 (Figure 12) gains 5.19 ERR and 2.93 F1, while Qwen3.5 (Figure 3) gains 5.28 ERR and 3.17 F1. The improvement for DeepSeekV4-Pro (Figure 14) is relatively smaller but still positive, with increases of 1.25 ERR and 0.72 F1. Averaged across all three models, threshold optimization improves ERR by +3.91 and F1 by +2.27. These gains are larger than those obtained by replacing the detector only in Ablation 1, identifying threshold optimization as the primary source of the final pipeline improvement.

Language	Baseline ERR/F1		Ablation2 ERR/F1		$\Delta \text{ERR} / \Delta \text{F1}$
DE	49.03	69.71	51.09	71.63	2.06 / 1.92
EN	56.56	76.64	57.35	77.76	0.79 / 1.12
HR	47.89	68.73	48.66	68.65	0.77 / -0.08
ID	62.76	77.69	62.96	77.91	<u>0.20 / 0.22</u>
IDEN	60.17	78.64	62.22	79.78	2.05 / 1.14
JA	14.35	37.43	26.06	46.10	<u>11.71 / 8.67</u>
KO	-9.04	20.96	7.83	19.90	<u>16.87 / -1.06</u>
NL	38.52	61.20	42.09	64.54	3.57 / 3.34
SL	54.62	73.30	58.51	76.81	3.89 / 3.51
SR	56.43	74.16	56.72	74.64	0.29 / 0.48
TH	24.05	50.78	38.42	65.23	<u>14.37 / 14.45</u>
VI	68.03	81.83	74.78	86.21	<u>6.75 / 4.38</u>
Avg Δ	-	-	+5.28	+3.17	-

Figure 3: Ablation 2 with Qwen3.5:9b: Effect of Threshold Optimization only

The effect is particularly obvious on Asian-language dataset. Averaged across the three mod-

els, threshold optimization improves ERR and F1 on the five Asian languages by 6.56 and 3.12, compared with 2.01 ERR and 1.67 F1 on the remaining non-Asian languages. Large ERR gains are observed for TH, JA, and VI. KO also obtains substantial ERR improvements across the Qwen models. ID receives only modest gains from threshold optimization when the original detector is used. These results indicate that language-specific threshold selection is especially important for languages whose normalization decisions are poorly served by a single fixed threshold.

The results of Ablation 2 are close to those of the complete pipeline (see 4.1), further confirming that threshold optimization accounts for most of the overall improvement. However, a comparison with the final results reveals non-negligible differences for individual languages and models. The largest absolute differences occur on JA and KO. With Qwen2.5-7B, adding the length-aware detector to the threshold-optimization further improves ERR by 2.49 on JA and 4.22 on KO. DeepSeekV4-Pro shows a similar trend, with additional ERR gains of 3.81 and 3.61 points. In contrast, Qwen3.5-9B shows reductions of 9.08 ERR points on JA and 7.83 points on KO. This indicates the detector-threshold interaction is model-dependent. Therefore, the length-aware detector does not uniformly increase the average benefit of threshold optimization. But its substantial effects on particular Asian languages (especially JA and KO) is not negligible. Overall, threshold optimization is the dominant source of improvement, while the length-aware detector provides a complementary contribution.

4.4 Ablation 3: Threshold Optimization contribution for new detector

Ablation 3 evaluates whether threshold optimization remains effective when applied to the length-aware detector. It compares the new detector using the default thresholds with the complete pipeline, in which the detector and entropy thresholds are optimized separately for each language. Threshold optimization produces clear average improvements across all three models. Qwen2.5 (15) gains 5.70 ERR and 3.00 F1, Qwen3.5 (4) gains 3.78 ERR and 2.85 F1, and DeepSeekV4-Pro (17) gains 1.32 ERR and 0.85 F1 points. Averaged across the three models, the improvements amount to 3.60 ERR and 2.24 F1. These results confirm that threshold optimization remains effective with the proposed

detector and is the primary source of the complete pipeline’s overall improvement. More gains are

Language	New Detector without specific thresholds ERR/F1		Ablation3 ERR/F1		Δ ERR/ Δ F1
DE	47.08	67.09	48.00	67.66	0.92 / 0.57
EN	58.45	77.95	60.82	79.50	2.37 / 1.55
HR	48.84	68.66	49.14	68.34	0.30 / -0.32
ID	63.78	78.32	64.03	78.58	0.25 / 0.26
IDEN	54.70	75.97	58.80	77.75	4.10 / 1.78
JA	16.98	37.62	16.98	37.62	0.00 / 0.00
KO	-6.02	14.56	0.00	17.82	6.02 / 3.26
NL	36.14	59.18	40.81	63.47	4.67 / 4.29
SL	54.16	72.95	58.18	76.56	4.02 / 3.61
SR	57.24	75.04	57.46	75.20	0.22 / 0.16
TH	17.56	50.08	33.46	64.97	15.90 / 14.89
VI	69.15	82.58	75.71	86.80	6.56 / 4.22
Avg Δ	-	-	+3.78	+2.85	-

Figure 4: Ablation 3 with Qwen3.5:9b: Effect of Threshold Optimization on new detector

also observed in the Asian language dataset. Across the three models, threshold optimization with the length-aware detector improves Asian-language ERR by an average of 5.60 and F1 by 3.24, compared with 2.17 ERR and 1.52 F1 on the remaining non-Asian languages. Particularly large improvements are observed on TH, VI, and KO. For example, Qwen2.5 gains 16.28 ERR on TH, 11.02 on VI, and 10.24 on KO. Qwen3.5 similarly gains 15.90 ERR and 14.89 F1 on TH, while DeepSeekV4-Pro obtains its largest Asian-language improvement on JA, with gains of 6.88 ERR and 4.74 F1.

A comparison with Ablation 2 provides a more nuanced view of the detector’s complementary contribution. There are positive improvement for two of the three models. Relative to threshold optimization with the original detector, the length-aware detector increases the Asian-language threshold gain of Qwen2.5-7B by 0.97 ERR and 0.53 F1 points, and that of DeepSeekV4-Pro by 0.39 ERR and 0.63 F1 points. In contrast, the corresponding gains for Qwen3.5-9B are reduced by 4.23 ERR and 0.81 F1 points. The positive improvements are also concentrated in particular language-model combinations. The length-aware detector produces a larger threshold-related ERR gain on TH for Qwen2.5 and on JA and KO for DeepSeekV4-Pro. This indicates that the detector provides a complementary rather than universal contribution: threshold optimization drives the overall performance improvement, while the length-aware representation can further strengthen it for specific Asian languages and downstream LLMs.

4.5 Ablation 4: Threshold Optimization contribution for both detectors

Ablation 4 directly compares the original and length-aware detectors after applying threshold optimization to both systems. The effect of replacing the detector is small and model-dependent. For Qwen2.5 (18), ERR improves by 0.95%, while F1 decreases slightly by 0.36%. DeepSeekV4-Pro (??) obtains positive but slight improvements of 0.65 ERR and 0.15 F1, while Qwen3.5 (5) decreases by 1.94 ERR and 1.24 F1. The Asian-language results are similarly heterogeneous: ID and VI improve across all three models, while the effects on JA and KO depend strongly on the downstream LLM. These findings indicate that the length-aware detector does not provide a universal improvement after threshold optimization and should instead be regarded as a complementary component whose benefit is specific to particular language-model combinations.

Language	Baseline with specific thresholds ERR/F1		Ablation4 ERR/F1		Δ ERR / Δ F1
DE	51.09	71.63	48.00	67.66	-3.09 / 16.57
EN	57.35	77.76	60.82	79.50	3.47 / 22.15
HR	48.66	68.65	49.14	68.34	0.48 / 19.68
ID	62.96	77.91	64.03	78.58	<u>1.07 / 15.62</u>
IDEN	62.22	79.78	58.80	77.75	-3.42 / 15.53
JA	26.06	46.10	16.98	37.62	<u>-9.08 / 11.56</u>
KO	7.83	19.90	0.00	17.82	<u>-7.83 / 9.99</u>
NL	42.09	64.54	40.81	63.47	-1.28 / 21.38
SL	58.51	76.81	58.18	76.56	-0.33 / 18.05
SR	56.72	74.64	57.46	75.20	0.74 / 18.48
TH	38.42	65.23	33.46	64.97	<u>-4.96 / 26.55</u>
VI	74.78	86.21	75.71	86.80	<u>0.93 / 12.02</u>
Avg Δ	-	-	-1.94	-1.24	-

Figure 5: Ablation 4 with Qwen3.5:9b: Effect of Threshold Optimization on both detectors

5 Conclusion

In this work, we reproduced the original normalization pipeline and introduced two main extensions: language-specific threshold optimization and a Length-Aware Detector. The complete pipeline improves the baseline by an average of 3.79 ERR and 1.79 F1 percentage points across the three evaluated LLMs. The gains are particularly pronounced for the Asian-language subset, although the magnitude varies across languages and models. Our ablation experiments show that threshold optimization is the primary source of the overall improvement. When applied to the original detector, it increases the average ERR and F1 by 3.91 and 2.27 points, respectively. In contrast, the Length-Aware Detector alone produces only marginal and inconsistent macro-level changes. Nevertheless,

it provides a complementary contribution for particular language-model combinations, especially for selected Asian languages. Overall, the results demonstrate the importance of language-specific calibration while suggesting that length-aware detection can provide additional benefits when combined with an appropriate downstream model and threshold configuration.

Limitations

The first limitation concerns the heterogeneous difficulty distribution across languages in the current dataset. In languages such as German, Slovenian, and Serbian, many tokens requiring normalization are spelling errors or common abbreviations. These cases can often be handled accurately using dictionary. In contrast, the Asian-language datasets, as well as English and IDEN, contain more informal expressions, internet slang, idiosyncratic usage and social-media noise, which includes usernames and URLs. Consequently, the evaluated languages differ not only linguistically but also in the types and difficulty of their normalization phenomena. These differences introduce language-dependent factors that are not fully captured by aggregate ERR and F1 comparisons. A second limitation is the relatively small amount of detector training data, particularly for the less frequent multiword normalization classes. This data scarcity and class imbalance can cause the Length-Aware Detector to misclassify one-to-one normalizations as one-to-many cases. Such errors may propagate to subsequent dictionary lookup and LLM processing. Finally, although the predicted length labels are incorporated into the LLM instructions, the in-context examples are not specifically constructed for NORM1WORD, NORM2WORD, and NORM3PLUS. The LLM may therefore be unable to fully exploit the intended length information. Future work could address these limitations through more balanced or augmented detector training data, label-specific in-context examples, improved handling of social-media noise, and evaluation protocols stratified by normalization phenomenon.

Contributions

Jiayang Zhang Individual contributions included reproducing the UFAL baseline experiments; designing, implementing and training the Length-Aware Detector; managing the LRZ computing environment for model training and final

evaluation; organizing and validating the final experimental results; writing approximately half of the final report, with primary responsibility for the Results and Appendix sections.

Yujiakuan Wang Yujiakuan Wang reproduced the original MultiLexNorm++ pipeline, implemented and ran the language-specific threshold search, and contributed part of the experimental results. He also wrote approximately half of the report, including the introduction, reproduced baseline pipeline, threshold optimization method, the whole experimental setup, and part of the appendix.

Code Our code is available at: https://github.com/Jiayang34/MultiLexNorm_LLM

References

Weerayuth Buaphet, Thong Nguyen Nguyen, Ryo Kondo, Tomoyuki Kajiwara, Younggyun Kim, John Lee, et al. 2026. [MultiLexNorm++: A unified benchmark and a generative model for lexical normalization for asian languages](#). ArXiv preprint.

Alexis Conneau, Kartikay Khandelwal, Naman Goyal, Vishrav Chaudhary, Guillaume Wenzek, Francisco Guzmán, Edouard Grave, Myle Ott, Luke Zettlemoyer, and Veselin Stoyanov. 2020. [Unsupervised cross-lingual representation learning at scale](#). In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 8440–8451, Online. Association for Computational Linguistics.

Bo Han and Timothy Baldwin. 2011. [Lexical normalization of short text messages: Makn sens a #twitter](#). In *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies*, pages 368–378, Portland, Oregon, USA. Association for Computational Linguistics.

Barbara Plank, Kristian Nørgaard Jensen, and Rob van der Goot. 2020. [DaN+: Danish nested named entities and lexical normalization](#). In *Proceedings of the 28th International Conference on Computational Linguistics*, pages 6649–6662, Barcelona, Spain (Online). International Committee on Computational Linguistics.

Rob van der Goot, Alan Ramponi, Arkaitz Zubiaga, Barbara Plank, Benjamin Muller, Iñaki San Vicente Roncal, Nikola Ljubešić, Özlem Çetinoğlu, Rahmad Mahendra, Talha Çolakoğlu, Timothy Baldwin, Tommaso Caselli, and Wladimir Sidorenko. 2021a. [MultiLexNorm: A shared task on multilingual lexical normalization](#). In *Proceedings of the Seventh Workshop on Noisy User-generated Text (W-NUT 2021)*, pages 493–509, Online. Association for Computational Linguistics.

Rob van der Goot, Ahmet Üstün, Alan Ramponi, Ibrahim Sharaf, and Barbara Plank. 2021b. [Massive choice, ample tasks \(MaChAmp\): A toolkit for multi-task learning in NLP](#). In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*, pages 176–197, Online. Association for Computational Linguistics.

Jianguo Zhang, Kazuma Hashimoto, Yao Wan, Zhiwei Liu, Ye Liu, Caiming Xiong, and Philip Yu. 2022. [Are pre-trained transformers robust in intent classification? a missing ingredient in evaluation of out-of-scope intent detection](#). In *Proceedings of the 4th Workshop on NLP for Conversational AI*, pages 12–20, Dublin, Ireland. Association for Computational Linguistics.

A Prompt Templates

Table 1 shows the LLM prompt templates for the original and length-aware detector settings.

B Detector Training Configuration

Table 2 summarizes the main MaChAmp configuration used for detector training.

C Appendix

C.1 Final results of new pipeline

Language	Baseline ERR/F1		New Pipeline ERR/F1		Δ ERR/ ΔF1
DE	32.42	55.90	37.00	58.21	4.58/2.31
EN	58.14	77.21	61.45	79.63	3.31/2.42
HR	36.72	59.95	40.52	61.66	3.80/1.71
ID	58.68	74.82	60.87	76.41	2.19/1.59
IDEN	58.46	77.35	59.83	78.26	1.37/0.91
JA	10.69	33.41	18.01	36.22	7.32/2.81
KO	-12.05	19.13	2.41	14.74	14.46/-4.39
NL	28.64	52.67	33.03	55.74	4.39/3.07
SL	44.83	66.17	54.25	72.53	9.42/6.36
SR	47.61	67.81	48.05	68.17	0.44/0.36
TH	23.79	53.67	34.73	59.70	10.94/6.03
VI	62.96	78.42	74.45	86.00	11.49/7.58
Avg Δ	-	-	+6.14	+2.56	-

Figure 6: Final result of new pipeline with Qwen2.5:7b

Language	Baseline		New Pipeline		Δ ERR/ ΔF1
DE	49.03	69.71	48.00	67.66	-1.03 / -2.05
EN	56.56	76.64	60.82	79.50	4.26 / 2.86
HR	47.89	68.73	49.14	68.34	1.25 / -0.39
ID	62.76	77.69	64.03	78.58	1.27 / 0.89
IDEN	60.17	78.64	58.80	77.75	-1.37 / -0.89
JA	14.35	37.43	16.98	37.62	2.63 / 0.19
KO	-9.04	20.96	0.00	17.82	9.04 / -3.14
NL	38.52	61.20	40.81	63.47	2.29 / 2.27
SL	54.62	73.30	58.18	76.56	3.56 / 3.26
SR	56.43	74.16	57.46	75.20	1.03 / 1.04
TH	24.05	50.78	33.46	64.97	9.41 / 14.19
VI	68.03	81.83	75.71	86.80	7.68 / 4.97
Avg Δ	-	-	+3.34	+1.93	-

Figure 7: Final result of new pipeline with Qwen3.5:9b

Original detector prompt	Length-aware detector prompt
Standardize the word or phrase enclosed in <MASK> by converting it to its formal and grammatically correct form. Consider the sentence context, punctuation, capitalization, and reduplication. Preserve the original meaning with minimal edits. If no normalization is needed, return the original word or phrase. Return only a JSON list with one string. <i>[8-shot in-context examples sampled from the same language are inserted here.]</i>	Standardize the word or phrase enclosed in <MASK> by converting it to its formal and grammatically correct form. Consider the sentence context, including punctuation, capitalization, and reduplication. Preserve the original meaning with minimal edits. The normalized output should contain exactly N whitespace-separated word(s) and keep them in the same string, e.g. ["want to"] . If no normalization is needed, return the original word or phrase. Return only a JSON list with one string. <i>[8-shot in-context examples sampled from the same language are inserted here.]</i>

Table 1: Prompt templates for the original and length-aware detector settings. The bracketed text describes omitted in-context examples.

Language	Baseline ERR/F1		New Pipeline ERR/F1		Δ ERR/ Δ F1
DE	52.92	72.10	50.74	69.68	-2.18 / -2.42
EN	61.77	79.83	63.35	80.89	1.58 / 1.06
HR	61.73	77.85	62.63	78.58	0.90 / 0.73
ID	68.74	81.92	71.46	83.80	<u>2.72 / 1.88</u>
IDEN	67.01	82.69	66.84	82.59	-0.17 / -0.10
JA	32.94	55.01	41.58	60.84	<u>8.64 / 5.83</u>
KO	-0.60	28.33	3.61	22.33	<u>4.21 / -6.00</u>
NL	52.24	71.54	53.89	73.47	1.65 / 1.93
SL	72.31	84.93	73.11	85.59	0.80 / 0.66
SR	66.94	81.03	68.92	82.81	1.98 / 1.78
TH	42.24	67.57	41.48	70.66	<u>-0.76 / 3.09</u>
VI	76.00	86.92	79.32	88.98	<u>3.32 / 2.06</u>
Avg Δ			+1.89	+0.88	-

Figure 8: Final result of new pipeline with DeepSeek-V4pro

Language	Baseline ERR/F1		Ablation1 ERR/F1		Δ ERR/ Δ F1
DE	52.92	72.10	52.58	71.33	-0.34 / -0.77
EN	61.77	79.83	63.35	80.86	1.58 / 1.03
HR	61.73	77.85	61.68	77.46	-0.05 / -0.39
ID	68.74	81.92	70.98	83.42	<u>2.24 / 1.50</u>
IDEN	67.01	82.69	66.84	82.92	-0.17 / 0.23
JA	32.94	55.01	34.70	56.10	<u>1.76 / 1.09</u>
KO	-0.60	28.33	1.81	22.75	<u>2.41 / -5.58</u>
NL	52.24	71.54	52.15	71.65	-0.09 / 0.11
SL	72.31	84.93	72.19	84.77	-0.12 / -0.16
SR	66.94	81.03	68.55	82.39	1.61 / 1.36
TH	42.24	67.57	39.69	68.98	<u>-2.55 / 1.41</u>
VI	76.00	86.92	76.61	87.34	<u>0.61 / 0.42</u>
Avg Δ	-	-	+0.57	+0.02	-

Figure 11: Ablation 1 with Deepseek-V4pro: Effect of length-aware detector only

Language	Baseline ERR/F1		Ablation1 ERR/F1		Δ ERR/ Δ F1
DE	32.42	55.90	32.65	54.63	0.23 / -1.27
EN	58.14	77.21	59.08	78.03	0.94 / 0.82
HR	36.72	59.95	38.03	60.02	1.31 / 0.07
ID	58.68	74.82	60.72	76.30	<u>2.04 / 1.48</u>
IDEN	58.46	77.35	58.12	77.71	-0.34 / 0.36
JA	10.69	33.41	14.06	33.37	<u>3.37 / -0.04</u>
KO	-12.05	19.13	-7.83	14.35	<u>4.22 / -4.78</u>
NL	28.64	52.67	27.81	51.86	-0.83 / -0.81
SL	44.83	66.17	45.17	66.32	0.34 / 0.15
SR	47.61	67.81	46.51	67.50	-1.10 / -0.31
TH	23.79	53.67	18.45	52.34	<u>-5.34 / -1.33</u>
VI	62.96	78.42	63.43	78.80	<u>0.47 / 0.38</u>
Avg Δ	-	-	+0.44	-0.44	-

Figure 9: Ablation 1 with Qwen2.5:7b: Effect of length-aware detector only

Language	Baseline ERR/F1		Ablation2 ERR/F1		Δ ERR/ Δ F1
DE	32.42	55.90	37.23	61.02	4.81 / 5.12
EN	58.14	77.21	58.93	78.37	0.79 / 1.16
HR	36.72	59.95	40.05	61.27	3.33 / 1.32
ID	58.68	74.82	58.77	74.90	<u>0.09 / 0.08</u>
IDEN	58.46	77.35	60.51	78.31	2.05 / 0.96
JA	10.69	33.41	15.52	35.67	<u>4.83 / 2.26</u>
KO	-12.05	19.13	-1.81	15.08	<u>10.24 / -4.05</u>
NL	28.64	52.67	33.12	57.03	4.48 / 4.36
SL	44.83	66.17	53.95	72.32	9.12 / 6.15
SR	47.61	67.81	48.57	68.61	0.96 / 0.80
TH	23.79	53.67	35.24	63.82	<u>11.45 / 10.15</u>
VI	62.96	78.42	73.14	85.24	<u>10.18 / 6.82</u>
Avg Δ	-	-	+5.19	+2.93	-

Figure 12: Ablation 2 with Qwen2.5:7b: Effect of Threshold Optimization only

Language	Baseline ERR/F1		Ablation1 ERR/F1		Δ ERR/ Δ F1
DE	49.03	69.71	47.08	67.09	-1.95 / -2.62
EN	56.56	76.64	58.45	77.95	1.89 / 1.31
HR	47.89	68.73	48.84	68.66	0.95 / -0.07
ID	62.76	77.69	63.78	78.32	<u>1.02 / 0.63</u>
IDEN	60.17	78.64	54.70	75.97	-5.47 / -2.67
JA	14.35	37.43	16.98	37.62	<u>2.63 / 0.19</u>
KO	-9.04	20.96	-6.02	14.56	<u>3.02 / -6.40</u>
NL	38.52	61.20	36.14	59.18	-2.38 / -2.02
SL	54.62	73.30	54.16	72.95	-0.46 / -0.35
SR	56.43	74.16	57.24	75.04	0.81 / 0.88
TH	24.05	50.78	17.56	50.08	<u>-6.49 / -0.70</u>
VI	68.03	81.83	69.15	82.58	<u>1.12 / 0.75</u>
Avg Δ	-	-	-0.44	-0.92	-

Figure 10: Ablation 1 with Qwen3.5:9b: Effect of length-aware detector only

Language	Baseline ERR/F1		Ablation2 ERR/F1		Δ ERR/ Δ F1
DE	49.03	69.71	51.09	71.63	2.06 / 1.92
EN	56.56	76.64	57.35	77.76	0.79 / 1.12
HR	47.89	68.73	48.66	68.65	0.77 / -0.08
ID	62.76	77.69	62.96	77.91	<u>0.20 / 0.22</u>
IDEN	60.17	78.64	62.22	79.78	2.05 / 1.14
JA	14.35	37.43	26.06	46.10	<u>11.71 / 8.67</u>
KO	-9.04	20.96	7.83	19.90	<u>16.87 / -1.06</u>
NL	38.52	61.20	42.09	64.54	3.57 / 3.34
SL	54.62	73.30	58.51	76.81	3.89 / 3.51
SR	56.43	74.16	56.72	74.64	0.29 / 0.48
TH	24.05	50.78	38.42	65.23	<u>14.37 / 14.45</u>
VI	68.03	81.83	74.78	86.21	<u>6.75 / 4.38</u>
Avg Δ	-	-	+5.28	+3.17	-

Figure 13: Ablation 2 with Qwen3.5:9b: Effect of Threshold Optimization only

Parameter	Value
Transformer model	xlm-roberta-base
Batch size	32
Max tokens	1024
Max input length	128
Dropout	0.2
Encoder weights updated	true
Number of epochs	20
Optimizer	AdamW
Learning rate	0.0001
Weight decay	0.01
Betas	(0.9, 0.99)
Random seed	8446
Main validation metric	macro-F1
Scheduler cut fraction	0.3
Scheduler decay factor	0.38
Discriminative fine-tuning	true
Gradual unfreezing	true
Keep top checkpoints	1

Table 2: Main MaChAmp training configuration for detector training.

Language	Baseline ERR/F1		Ablation2 ERR/F1		Δ ERR/ Δ F1
DE	52.92	72.10	53.61	72.91	0.69 / 0.81
EN	61.77	79.83	60.51	79.54	-1.26 / -0.29
HR	61.73	77.85	61.91	77.96	0.18 / 0.11
ID	68.74	81.92	69.13	82.27	<u>0.39 / 0.35</u>
IDEN	67.01	82.69	68.38	83.61	1.37 / 0.92
JA	32.94	55.01	37.77	59.72	<u>4.83 / 4.71</u>
KO	-0.60	28.33	0.00	23.15	<u>0.60 / -5.18</u>
NL	52.24	71.54	52.61	72.03	0.37 / 0.49
SL	72.31	84.93	73.40	85.82	1.09 / 0.89
SR	66.94	81.03	67.74	81.87	0.80 / 0.84
TH	42.24	67.57	45.04	70.69	<u>2.80 / 3.12</u>
VI	76.00	86.92	79.09	88.80	<u>3.09 / 1.88</u>
Avg Δ	-	-	+1.25	+0.72	-

Figure 14: Ablation 2 with DeepSeek-V4pro: Effect of Threshold Optimization only

C.2 Ablation 1

C.3 Ablation2

C.4 Ablation 3

C.5 Ablation 4

Language	New Detector without specific thresholds ERR/F1		Ablation3 ERR/F1		Δ ERR/ Δ F1
DE	32.65	54.63	37.00	58.21	4.35 / 3.58
EN	59.08	78.03	61.45	79.63	2.37 / 1.60
HR	38.03	60.02	40.52	61.66	2.49 / 1.64
ID	60.72	76.30	60.87	76.41	<u>0.15 / 0.11</u>
IDEN	58.12	77.71	59.83	78.26	1.71 / 0.55
JA	14.06	33.37	18.01	36.22	<u>3.95 / 2.85</u>
KO	-7.83	14.35	2.41	14.74	<u>10.24 / 0.39</u>
NL	27.81	51.86	33.03	55.74	5.22 / 3.88
SL	45.17	66.32	54.25	72.53	9.08 / 6.21
SR	46.51	67.50	48.05	68.17	1.54 / 0.67
TH	18.45	52.34	34.73	59.70	<u>16.28 / 7.36</u>
VI	63.43	78.80	74.45	86.00	<u>11.02 / 7.20</u>
Avg Δ	-	-	+5.70	+3.00	-

Figure 15: Ablation 3 with Qwen2.5:7b: Effect of Threshold Optimization on new detector

Language	New Detector without specific thresholds ERR/F1		Ablation3 ERR/F1		Δ ERR/ Δ F1
DE	47.08	67.09	48.00	67.66	0.92 / 0.57
EN	58.45	77.95	60.82	79.50	2.37 / 1.55
HR	48.84	68.66	49.14	68.34	0.30 / -0.32
ID	63.78	78.32	64.03	78.58	<u>0.25 / 0.26</u>
IDEN	54.70	75.97	58.80	77.75	4.10 / 1.78
JA	16.98	37.62	16.98	37.62	<u>0.00 / 0.00</u>
KO	-6.02	14.56	0.00	17.82	<u>6.02 / 3.26</u>
NL	36.14	59.18	40.81	63.47	4.67 / 4.29
SL	54.16	72.95	58.18	76.56	4.02 / 3.61
SR	57.24	75.04	57.46	75.20	0.22 / 0.16
TH	17.56	50.08	33.46	64.97	<u>15.90 / 14.89</u>
VI	69.15	82.58	75.71	86.80	<u>6.56 / 4.22</u>
Avg Δ	-	-	+3.78	+2.85	-

Figure 16: Ablation 3 with Qwen3.5:9b: Effect of Threshold Optimization on new detector

Language	New Detector without specific thresholds ERR/F1		Ablation3 ERR/F1		Δ ERR/ Δ F1
DE	52.58	71.33	50.74	69.68	-1.84 / -1.65
EN	63.35	80.86	63.35	80.89	0.00 / 0.03
HR	61.68	77.46	62.63	78.58	0.95 / 1.12
ID	70.98	83.42	71.46	83.80	<u>0.48 / 0.38</u>
IDEN	66.84	82.92	66.84	82.59	0.00 / -0.33
JA	34.70	56.10	41.58	60.84	<u>6.88 / 4.74</u>
KO	1.81	22.75	3.61	22.33	<u>1.80 / -0.42</u>
NL	52.15	71.65	53.89	73.47	1.74 / 1.82
SL	72.19	84.77	73.11	85.59	0.92 / 0.82
SR	68.55	82.39	68.92	82.81	0.37 / 0.42
TH	39.69	68.98	41.48	70.66	<u>1.79 / 1.68</u>
VI	76.61	87.34	79.32	88.98	<u>2.71 / 1.64</u>
Avg Δ	-	-	+1.32	+0.85	-

Figure 17: Ablation 3 with DeepSeek-V4pro: Effect of Threshold Optimization on new detector

Language	Baseline with specific thresholds ERR/F1		Ablation4 ERR/F1		Δ ERR/ Δ F1
DE	37.23	61.02	37.00	58.21	-0.23 / -2.81
EN	58.93	78.37	61.45	79.63	2.52 / 1.26
HR	40.05	61.27	40.52	61.66	0.47 / 0.39
ID	58.77	74.90	60.87	76.41	<u>2.10 / 1.51</u>
IDEN	60.51	78.31	59.83	78.26	-0.68 / -0.05
JA	15.52	35.67	18.01	36.22	<u>2.49 / 0.55</u>
KO	-1.81	15.08	2.41	14.74	<u>4.22 / -0.34</u>
NL	33.12	57.03	33.03	55.74	-0.09 / -1.29
SL	53.95	72.32	54.25	72.53	0.30 / 0.21
SR	48.57	68.61	48.05	68.17	-0.52 / -0.44
TH	35.24	63.82	34.73	59.70	<u>-0.51 / -4.12</u>
VI	73.14	85.24	74.45	86.00	<u>1.31 / 0.76</u>
Avg Δ	-	-	+0.95	-0.36	-

Figure 18: Ablation 4 with Qwen2.5:7b: Effect of Threshold Optimization on both detectors

Language	Baseline with specific thresholds ERR/F1		Ablation4 ERR/F1		Δ ERR/ Δ F1
DE	51.09	71.63	48.00	67.66	-3.09 / 16.57
EN	57.35	77.76	60.82	79.50	3.47 / 22.15
HR	48.66	68.65	49.14	68.34	0.48 / 19.68
ID	62.96	77.91	64.03	78.58	<u>1.07 / 15.62</u>
IDEN	62.22	79.78	58.80	77.75	-3.42 / 15.53
JA	26.06	46.10	16.98	37.62	<u>-9.08 / 11.56</u>
KO	7.83	19.90	0.00	17.82	<u>-7.83 / 9.99</u>
NL	42.09	64.54	40.81	63.47	-1.28 / 21.38
SL	58.51	76.81	58.18	76.56	-0.33 / 18.05
SR	56.72	74.64	57.46	75.20	0.74 / 18.48
TH	38.42	65.23	33.46	64.97	<u>-4.96 / 26.55</u>
VI	74.78	86.21	75.71	86.80	<u>0.93 / 12.02</u>
Avg Δ	-	-	-1.94	-1.24	-

Figure 19: Ablation 4 with Qwen3.5:9b: Effect of Threshold Optimization on both detectors

Language	Baseline with specific thresholds ERR/F1		Ablation4 ERR/F1		Δ ERR/ Δ F1
DE	53.61	72.91	50.74	69.68	-2.87 / -3.23
EN	60.51	79.54	63.35	80.89	2.84 / 1.35
HR	61.91	77.96	62.63	78.58	0.72 / 0.62
ID	69.13	82.27	71.46	83.80	<u>2.33 / 1.53</u>
IDEN	68.38	83.61	66.84	82.59	-1.54 / -1.02
JA	37.77	59.72	41.58	60.84	<u>3.81 / 1.12</u>
KO	0.00	23.15	3.61	22.33	<u>3.61 / -0.82</u>
NL	52.61	72.03	53.89	73.47	1.28 / 1.44
SL	73.40	85.82	73.11	85.59	-0.29 / -0.23
SR	67.74	81.87	68.92	82.81	1.18 / 0.94
TH	45.04	70.69	41.48	70.66	<u>-3.56 / -0.03</u>
VI	79.09	88.80	79.32	88.98	<u>0.23 / 0.18</u>
Avg Δ	-	-	+0.65	+0.15	-

Figure 20: Ablation 4 with DeepSeek-V4pro: Effect of Threshold Optimization on both detectors